

RAG FDS

Grupo I - SIKA

11

Módulos Python

12

Scripts

10

Docs técnicos

Informe Final - RAG_FDS Grupo I (SIKA)

1. Objetivo y alcance

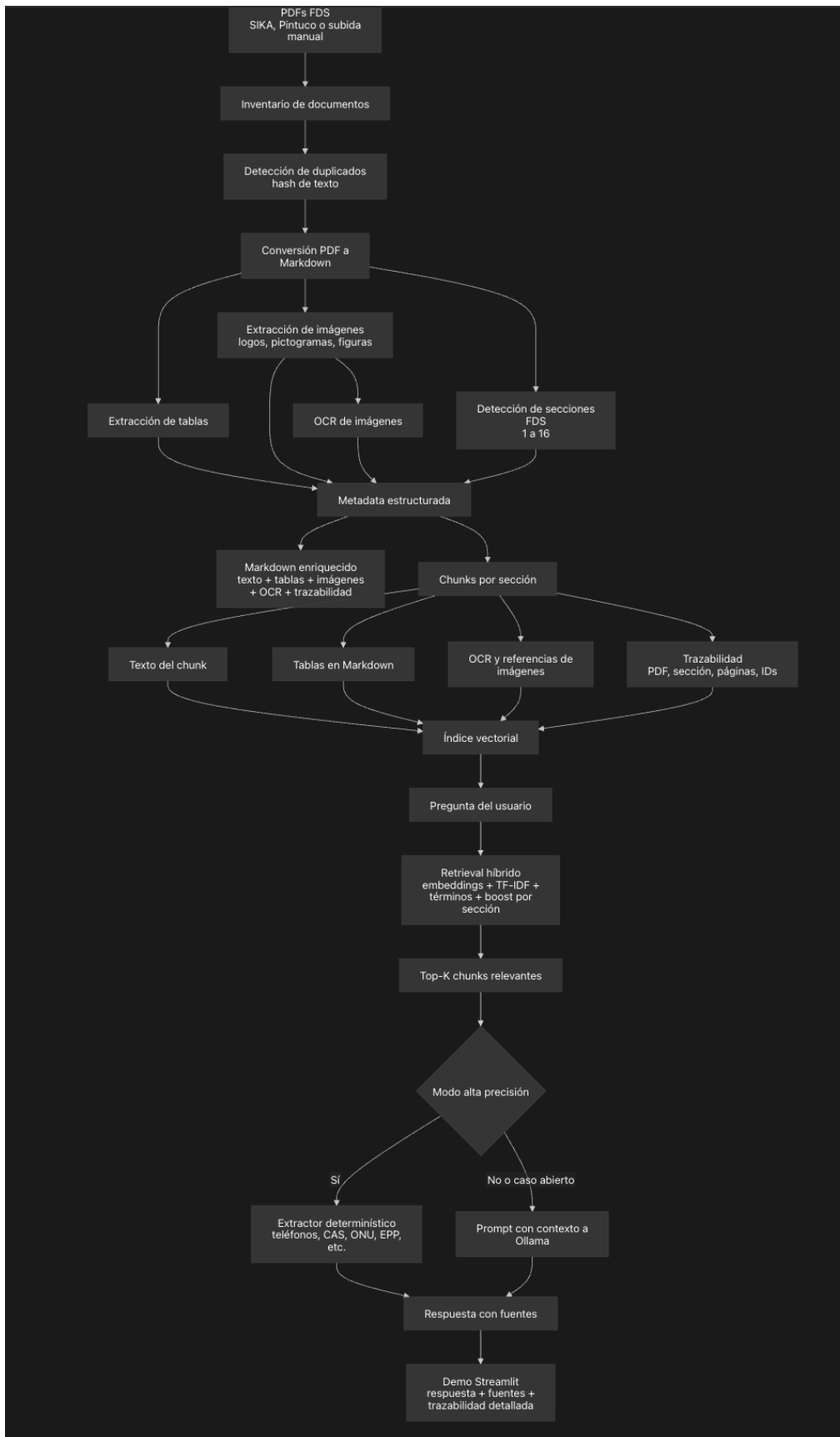
Se implemento un sistema RAG para Fichas de Datos de Seguridad (FDS) del fabricante asignado al Grupo I: **SIKA**.

El sistema cubre extraccion documental, conversion a Markdown, validacion estructural, chunking, indexacion, recuperacion y generacion con trazabilidad.

Se incluyo soporte de fabricante adicional y carga manual de PDFs en la demo para mostrar generalizacion.

2. Arquitectura final implementada

1. Extraccion PDF local con PyMuPDF + pypdf.
2. Deteccion y validacion de las 16 secciones normativas.
3. Extraccion de tablas y conversion a Markdown.
4. Extraccion de imagenes/pictogramas/logos + OCR local (spa+eng).
5. Asociacion estructurada de tablas/imagenes a secciones por pagina/proximidad.
6. Chunking por seccion con metadatos y trazabilidad.
7. Indice hibrido local (embeddings + TF-IDF + cobertura de terminos + boost por seccion).
8. Generacion con Ollama local (qwen2.5:3b) sobre contexto recuperado.
9. Demo Streamlit con corpus preprocesado y carga manual de PDFs.



Runtime final:

- OLLAMA_BASE_URL=http://127.0.0.1:11434
- OLLAMA_MODEL=qwen2.5:3b
- source scripts/env_local.sh para fijar variables locales del runtime.
- Servicio local validado con brew services start ollama.

3. Cumplimiento de requerimientos de la guía

- Pipeline PDF -> Markdown: **cumplido**.
- Preservacion estructural (titulos, tablas, listas, notas): **cumplido**.
- Identificacion de 16 secciones: **cumplido para corpus canonico SIKA**.
- Tratamiento de imagenes y OCR local: **cumplido**.
- Asociacion imagen-seccion-tabla y nota de trazabilidad: **cumplido**.
- Sistema RAG funcional con modelos/infraestructura propia: **cumplido**.
- Estrategia de chunking documentada: **cumplido**.
- Consultas con trazabilidad a fuente: **cumplido**.
- Ground truth y evaluacion comparativa: **cumplido con dataset y script de evaluacion**.

4. Resultados de procesamiento y calidad

Resumen principal del corpus SIKA (ver reportes en data/reports):

- 14 documentos canonicos procesados.
- 224 secciones normativas validadas.
- 39 tablas extraidas.
- 174 imagenes extraidas con OCR local.
- 224 chunks RAG validados.

Pruebas de retrieval previas:

- 5/5 consultas de control con seccion correcta en top-1.

Validacion automatica final:

- Conversion SIKA: 14/14 documentos pasan, calidad promedio 100.0.
- Chunking: 224/224 secciones cubiertas, 39/39 tablas cubiertas, 174/174 imagenes cubiertas.
- Retrieval smoke test: 5/5 top-1 correcto.

5. Ground truth y evaluacion RAG

Se construyo un set de 25 pares pregunta-respuesta:

- Archivo: data/ground_truth/sika_ground_truth_25.json.
- Cubre preguntas factuales, tecnicas y de trazabilidad.
- Incluye seccion esperada y terminos esperados por pregunta.

Script de evaluacion:

- scripts/evaluate_ground_truth.py
- Modo retrieval_only (estable, sin dependencia de red/LLM).
- Modo with_llm (evalua tambien respuesta generada).

Salidas:

- data/reports/ground_truth_eval_sika_retrieval_only.csv|json
- data/reports/ground_truth_eval_sika_with_llm.csv|json

Resultados finales:

```
{
  "retrieval_only": {
    "ground_truth_items": 25,
    "top1_section_accuracy": 1.0,
    "topk_section_recall": 1.0,
    "context_term_coverage_pass_rate": 1.0
  },
  "with_ollama": {
    "ground_truth_items": 25,
    "llm_answers_evaluated": 25,
    "top1_section_accuracy": 1.0,
    "topk_section_recall": 1.0,
    "context_term_coverage_pass_rate": 1.0,
    "answer_term_pass_rate": 1.0
  }
}
```

6. Limitaciones observadas

1. Las preguntas abiertas que requieren criterio no definido por la FDS (por ejemplo, "el quimico mas peligroso") no deben responderse como hecho documental.
2. La extraccion OCR puede registrar "OCR sin texto legible" cuando una imagen no contiene texto interpretable.
3. Las consultas comparativas amplias pueden requerir dividir la pregunta por seccion para mantener trazabilidad estricta.

7. Mitigaciones implementadas

1. Ollama local como runtime principal para eliminar dependencia de ngrok en la entrega.
2. Streaming de respuesta con `/api/generate`.
3. Reintentos automáticos y degradación controlada de contexto.
4. Extracción literal para teléfonos, CAS, ONU, EPP, imágenes/OCR y trazabilidad de sección.
5. Regla de no alucinación: sin evidencia suficiente, respuesta explícita de insuficiencia.

8. Demostración funcional

App Streamlit:

- Archivo: `streamlit_app.py`
- Funciones:
 - consulta sobre corpus preprocesado,
 - carga manual de PDFs y reprocesamiento en vivo,
 - respuesta con fuentes y trazabilidad detallada.

Mapa de requisitos vs. ubicación en el repo

Cada requisito del documento RAG_FDS.pdf mapeado a los archivos y carpetas donde se implementa.

Requisito	Estado	Ubicación en el proyecto
Extracción y conversión documental (20%)		
Pipeline PDF → Markdown	Cumple	<code>src/rag_fds/pdf_utils.py</code> <code>src/rag_fds/markdown_converter.py</code> <code>scripts/convert_pdfs.py</code> → <code>data/processed/markdown/SIKA/*.md</code> → <code>data/processed/markdown/Pintuco/*.md</code>
Detección de 16 secciones	Cumple	<code>src/rag_fds/sections.py</code> <code>scripts/validate_conversion.py</code> → <code>data/reports/quality_report_sika.csv</code>
OCR de imágenes (Tesseract)	Cumple	<code>src/rag_fds/structured_extraction.py</code> └ <code>ocr_image_bytes()</code> → <code>data/processed/assets/SIKA/*/images/</code>
Extracción de tablas	Cumple	<code>src/rag_fds/structured_extraction.py</code> └ <code>extract_tables()</code> con PyMuPDF → Embebidas en <code>.md</code> y metadata JSON
Fidelidad estructural del .md (30%)		
Estructura preservada	Cumple	Front matter YAML + tabla de secciones Títulos, subtítulos, tablas, listas, notas → <code>data/processed/markdown/SIKA/*.md</code>
Asociación imagen↔sección	Cumple	<code>structured_extraction.py</code> └ <code>section_for_bbox()</code>

Deduplicación de documentos	Cumple	Nota de trazabilidad debajo de cada imagen src/rag_fds/inventory.py └─ add_duplicate_flags() → data/reports/inventory.csv
Arquitectura y eficiencia (20%)		
Sin APIs pagas / open source	Cumple	PyMuPDF, pypdf, Tesseract, Ollama sentence-transformers (local) requirements.txt
Justificación de decisiones	Cumple	PLANTEAMIENTO_PROYECTO.md docs/INFORME_FINAL.md docs/CUMPLIMIENTO_GUIA.md
Modelo local / infra propia	Cumple	scripts/env_local.sh Ollama qwen2.5:3b @ localhost:11434 Embeddings: paraphrase-multilingual-MiniLM-L12-v2
Implementación del RAG (10%)		
Chunking documentado	Cumple	src/rag_fds/chunking.py docs/FASE_3_CHUNKING.md → data/processed/chunks/sika_chunks.jsonl
Índice vectorial + retriever	Cumple	src/rag_fds/vector_index.py → data/indexes/sika/ (embeddings.npy, tfidf_*.joblib) docs/FASE_4_INDICE_RETRIEVER.md
Generación con trazabilidad	Cumple	src/rag_fds/generation.py src/rag_fds/fact_extraction.py docs/FASE_5_GENERACION_OLLAMA.md
Evaluación ground truth	Cumple	data/ground_truth/sika_ground_truth_25.json scripts/evaluate_ground_truth.py → data/reports/ground_truth_eval_sika_*.json
Documentación y presentación (20%)		
Repositorio del proyecto	Cumple	README.md, .gitignore, requirements.txt
Documentación técnica	Cumple	docs/FASE_1 a FASE_5 docs/CONTROL_CALIDAD.md
Arquitectura general	Cumple	PLANTEAMIENTO_PROYECTO.md (Mermaid) docs/INFORME_FINAL.md
Ejemplos .md convertidos	Cumple	data/processed/markdown/SIKA/ (14 archivos) data/processed/markdown/Pintuco/ (19 archivos)
Demo funcional del RAG	Cumple	streamlit_app.py scripts/ask.py
Informe de limitaciones	Cumple	docs/INFORME_FINAL.md docs/CUMPLIMIENTO_GUIA.md

Estructura de desarrollo del proyecto

Árbol del repositorio organizado por capas funcionales. Cada archivo con su rol en el pipeline.

Código fuente	Scripts	Datos	Documentación	Aplicación	Configuración
---------------	---------	-------	---------------	------------	---------------

Código fuente (librería)

```
src/rag_fds/
├── __init__.py  versión del paquete
├── config.py    rutas y fabricantes
├── pdf_utils.py lectura PDF + hashes SHA-256
├── sections.py  detección de 16 secciones normativas
├── structured_extraction.py  tablas + imágenes + OCR Tesseract
├── markdown_converter.py  PDF → Markdown completo
├── inventory.py  inventario del corpus + duplicados
├── chunking.py   fragmentación por sección (850 tok)
├── vector_index.py  embeddings + TF-IDF + búsqueda híbrida
├── fact_extraction.py  extracción literal (CAS, ONU, EPP...)
├── generation.py  prompt + llamada Ollama
```

Scripts ejecutables

```
scripts/
├── run_inventory.py  Fase 1: diagnóstico del corpus
├── convert_pdfs.py   Fase 2: conversión PDF→MD
├── validate_conversion.py  QA de .md generados
├── build_chunks.py   Fase 3: chunking
├── validate_chunks.py  QA de chunks
├── build_vector_index.py  Fase 4: índice vectorial
├── retrieve.py        consulta al retriever
├── test_retrieval.py  test del retriever
├── ask.py             Fase 5: pregunta CLI
├── evaluate_ground_truth.py  evaluación GT (25 pares)
├── env_local.sh       variables de entorno Ollama
├── setup_grok_ollama.py  alternativa remota (anexo)
```

Datos generados

```
data/
├── processed/
│   ├── markdown/  SIKA/ Pintuco/ → archivos .md
│   ├── metadata/  SIKA/ Pintuco/ → JSON por doc
│   ├── assets/    SIKA/ Pintuco/ → imágenes extraídas
│   └── chunks/    sika_chunks.jsonl, pintuco_chunks.jsonl
```

```

|— indexes/
|   |— sika/ embeddings.npy + tfidf + config
|   |— pintuco/ embeddings.npy + tfidf + config
|— ground_truth/ sika_ground_truth_25.json
|— reports/ inventory, conversion, chunks, QA, eval

```

Documentación técnica

```

docs/
|— FASE_1_DIAGNOSTICO.md inventario del corpus
|— FASE_2_CONVERSION_MARKDOWN.md pipeline PDF→MD
|— FASE_3_CHUNKING.md estrategia de fragmentación
|— FASE_4_INDICE_RETRIEVER.md índice + búsqueda
|— FASE_5_GENERACION_OLLAMA.md generación con LLM
|— INFORME_FINAL.md resumen ejecutivo
|— CUMPLIMIENTO_GUIA.md check vs. enunciado
|— CONTROL_CALIDAD.md validaciones automáticas
|— NGROK_OLLAMA_RUNBOOK.md alternativa remota
|— SAGEMAKER_OLLAMA_SETUP.md anexo histórico

```

Aplicación y configuración (raíz)

```

streamlit_app.py demo interactiva RAG
README.md guía de uso del proyecto
PLANTEAMIENTO_PROYECTO.md diseño y arquitectura
requirements.txt dependencias Python
RAG_FDS.pdf enunciado del parcial
SageMaker_Ollama_Ngrok_Setup.ipynb notebook alternativo
Documentos - Parcial final/ PDFs fuente (SIKA, Pintuco, Corona)

```

9. Conclusiones

El proyecto implementa una arquitectura RAG reproducible y portable alineada con la guía RAG_FDS.

Se priorizo fidelidad documental y trazabilidad sobre respuestas especulativas.

La version final usa Ollama local como runtime principal y valida el sistema contra 25 preguntas ground truth con resultados completos.